Portfolio Design Engineering Review

A structural improvement specification and implementation workflow for the Miebaka Joshua portfolio

Main decision. The restored production website is the visual authority. The work below strengthens proof, navigation, accessibility, conversion and engineering quality without reviving the rejected redesign.

Date	13 October 2026
Production baseline	Commit 54dcc41 and miebaka-portfolio.vercel.app
Repository	github.com/miebaka/portfolio
Scope	Design engineering, frontend, backend, accessibility, performance, security and delivery

Evidence base. Live-site inspection, repository and commit analysis, both supplied Markdown documents, automated tests, and independent frontend and backend specialist audits.

Status: Approved direction for planning

Date: 13 October 2026

Visual authority: The restored production site at commit `54dcc41`

Supporting sources: `design-thesis.md` version 5.0, owner answers, production code, deployed site, frontend audit, backend audit

Decision

The current production website is the visual baseline. Its editorial restraint, typographic character, Ultramarine identity, image treatment, spacing, sharp geometry and quiet interaction language must be preserved. This work is not a revival of the rejected thesis-aligned redesign.

The design thesis remains useful as a requirements inventory, but it is subordinate wherever it conflicts with the restored site's visual character. Structural changes are allowed only when they improve proof, navigation, accessibility, conversion or maintainability without making the site busier or more generic.

Current state

The live deployment, local checkout and `origin/main` match commit `54dcc41`. The repository builds 14 pages and passes 20 integrity tests. Production uses one bundled stylesheet, a small ES-module graph, responsive image variants, strong security headers and no moderate-or-higher dependency vulnerabilities.

The green build does not mean the portfolio is complete. Several tests explicitly protect an older product contract: locked case studies, missing Explorations and Community routes, the old navigation model and incomplete conversion behavior.

Design preservation contract

The following qualities are protected:

- The present Ultramarine, paper and ink visual world.
- The current editorial typography and typographic tension.
- The spacious composition and low visual density.
- Sharp surfaces and restrained functional rounding.
- Authored imagery without interface-driven recolouring.
- Quiet motion that follows user action.
- The current portrait treatment and general photographic tone.
- The existing footer signature and overall page atmosphere.
- The site’s warm, direct and professionally exact language.

The implementation must not introduce dashboard composition, generic component-library styling, decorative cards, excessive borders, bubbly Material geometry, unrelated accent colours, gratuitous motion or dense proof modules.

Primary problems

The work cannot currently be evaluated

Every professional case study is locked. Homepage project cards are non-interactive articles and the Work route exposes disabled “Coming soon” states. The site therefore claims senior design capability without letting hiring managers inspect a complete project.

TEDx RSU must become the first complete flagship case study. Its structure should be adapted to the current site's editorial language rather than copied mechanically from the design thesis.

The information architecture is incomplete

Explorations and Community are absent. The restored navigation is visually coherent but does not provide the expanded route system requested by the owner. New destinations must be introduced with the same typographic and spatial restraint as existing navigation.

The homepage does not build proof in the strongest order

The current sequence places the personal introduction before selected work. Carefully moving selected work earlier would let the portfolio establish competence before biography. This change should reuse the existing sections and treatments rather than invent a new homepage system.

Contact is not a functioning web conversion

The contact form validates only name and email, does not require a message and ultimately opens a `mailto:` URL. The production `/api/contact` route returns 404. The primary conversion therefore depends on a configured mail client and cannot report delivery success or failure.

Tests validate the wrong contract

The current tests pass because they require locked case studies and the restored route set. Tests must become an executable expression of the approved visual baseline plus the new structural requirements.

Proposed information architecture

Primary destinations

- Home
- Work
- Explorations
- Community
- About
- Services
- Writing
- Contact

Privacy remains a footer destination. CV download appears within About and the expanded navigation rather than as a top-level page.

Navigation behavior

Desktop keeps the current visual treatment. It gains an Index entry that opens a restrained full-screen editorial layer containing the complete destination set, contact action, CV action and preferences. The persistent bar should not become crowded.

Compact layouts retain a small, direct control model. A permanent five-item bottom bar should be introduced only if measured testing proves it improves wayfinding without competing with content. The default recommendation is to keep the current compact control and strengthen the Index layer first.

Homepage order

1. Existing full-screen proposition.
2. Selected work using the current project-card language.
3. Existing profile and portrait section.
4. Services using a refined form of the current presentation.
5. Client proof with an operable pause control.
6. Working principles, only after owner-authored copy exists.
7. Existing contact invitation.
8. Existing signature and footer.

The exact support line from the thesis may be added to the hero only if it can be integrated without weakening the restored composition. It is not automatically required.

Page changes

Home

- Move selected work before the profile section.
- Convert publication-ready projects into real links.
- Keep unfinished work visibly distinct and non-deceptive.
- Preserve current imagery, typography and section vocabulary.
- Add a visible Pause or Play control to the moving client marks.
- Increase small body and interface type only where measured readability requires it.
- Add working principles only when real copy is supplied.

Work

- Replace the universal locked state with mixed publishing states.
- Present available case studies as direct entries.
- Keep unfinished projects listed only when their presence helps the portfolio and their state is clear.
- Preserve the existing editorial disclosure rhythm where it remains useful.

TEDx RSU case study

- Publish the project proposition, hero media, role, collaborators, year and deliverables.
- Explain context, central problem, decision, system, applications, production, verified outcomes and reflection.
- Show process that led to the final direction; do not publish rejected routes.
- Use inline media inside the existing editorial measure.

- End with one next-project transition.
- Add restrained chapter orientation without turning the page into an application interface.

Remaining case studies

- Keep locked until enough authored media and verified evidence exist.
- Use the same data contract as TEDx RSU so later publication does not require a new template.

Explorations

- Create a real route using the site's existing visual grammar.
- Each entry needs a self-initiated prompt, visual decision and date.
- If no entry is ready, publish a truthful introductory empty state rather than invented cards.

Community

- Create a real route with the same restraint.
- Each entry identifies audience, contribution and access method.
- Downloads identify format, licence, version and last-updated date.
- A truthful empty state is acceptable until real material exists.

About

- Retain the current portrait and tone.
- Verify every numerical or outcome claim.
- Add a stable `DOWNLOAD CV` action with PDF format and file size.
- Remove or qualify unsupported claims.

Services

- Preserve the present service language unless owner-supplied revisions improve precision.
- Ensure the homepage and Services page use one source of truth.
- Avoid decorative service cards or unrelated imagery.

Writing

- Keep the name Writing.
- Use only factual publication date and reading length metadata.
- Preserve the current editorial article design.

Contact

- Keep the visible email address and copy action.
- Make message required.
- Submit progressively to a same-origin serverless endpoint.
- Preserve entered values on failure.
- Provide clear submitting, success, validation, rate-limit and provider-failure states.
- Keep a truthful no-JavaScript email fallback.

Privacy

- Update the page when server submission is introduced.
- State which fields are collected, why, which processors deliver the message and how long operational data is retained.
- Disclose Last.fm behavior if listening status remains client-side.

404

- Retain the current special treatment only if it stays inside performance and reduced-capability budgets.

Frontend engineering architecture

Content

Split `data/site.js` into validated public domains for site configuration, navigation, projects, case studies, services, writing and client marks. A build-time schema must reject missing slugs, missing media dimensions, invalid project states, duplicate routes and broken next-project references.

Rendering

Core narrative content must be prerendered. JavaScript enhances navigation, animation, media and live status but does not own the only copy of essential sections.

Styles

Consolidate the repeated override strata in `styles/pages-v4.css`. Keep one production CSS request, but give source files clear responsibilities: foundations, components, pages and route-specific enhancements. Replace raw interface colours with semantic tokens while leaving authored project artwork untouched.

Interaction

Pointer-only transformations must be gated behind fine-pointer hover capability. Keyboard and touch receive equivalent information and state without surprise geometry changes. Reduced motion removes continuous and spatial movement while retaining concise colour and opacity feedback.

Responsive behavior

Support 320px through large desktop. Every route must be verified for horizontal overflow, 200 percent zoom, text spacing, safe-area insets, touch targets and both themes.

Backend architecture

Keep the site static-first. Add one narrow contact boundary:

```
HTTP handler -> request validator -> abuse guard -> mail-service interface -> Resend adapter -> structured response and safe logger
```

No database, authentication system or broad backend framework is justified.

The endpoint must:

- Accept only same-origin JSON POST requests.
- Enforce request and field-size limits.
- Require valid name, email and message values.
- Reject unexpected interest values and malformed bodies.
- Use an accessible honeypot and minimum-fill-time signal.
- Apply shared infrastructure-backed rate limiting rather than process memory.
- Bound provider work with an eight-second timeout.
- Return stable error codes for validation, abuse, provider failure and internal failure.
- Log request ID, status, latency and provider response class without logging message bodies or email addresses.

Accessibility requirements

- WCAG 2.2 AA is the minimum; existing higher-contrast placements should be retained.
- Do not impose a universal 7:1 requirement where it damages the approved visual baseline without measurable benefit.
- All controls need visible focus and keyboard operation.
- Targets must provide a practical 44 by 44 pixel interaction region.
- The marquee requires a visible stop control.
- Form errors must be associated without overwriting existing descriptions.
- The site must remain understandable with reduced motion, forced colours, increased text size and JavaScript unavailable.

Performance and resilience

- LCP target at or below 2.0 seconds; hard gate 2.5 seconds.
- INP target at or below 150 milliseconds; hard gate 200 milliseconds.
- CLS at or below 0.05.
- Initial compressed JavaScript at or below 75KB.
- Initial compressed CSS at or below 45KB.
- Homepage media before interaction at or below 1.5MB.
- No ordinary navigation long task above 200 milliseconds.
- No non-essential idle main-thread work.
- Use content-hashed filenames before assigning one-year immutable cache headers.
- Save-Data and reduced-motion environments receive quieter media behavior.

Security and privacy

- Keep the current CSP and security-header baseline.
- Make Vercel the canonical deployed configuration and test Netlify parity only if Netlify remains supported.
- Remove inline-style dependencies where practical before tightening `style-src`.
- Introduce CSP reporting in report-only mode before enforcement changes.
- Restrict and document the public Last.fm key.
- Consider a cached server proxy for Last.fm if the listening feature survives review.

Verification

Automated tests must cover content schemas, route generation, required homepage order, mixed project states, the published TEDx chapter contract, navigation, CV delivery, form validation, provider mocking, rate limiting, origin checks, timeout behavior, value retention and privacy-copy parity.

Browser verification must cover all production routes at 320px, 390px, tablet, 1280px and 1440px; Print and Screen themes; keyboard, touch and pointer; reduced motion; forced colours; 200 percent zoom; text-spacing overrides; JavaScript disabled; console errors; broken assets; Lighthouse budgets and production headers.

Delivery order

9. Freeze and test the visual preservation contract.
10. Replace obsolete product-contract tests.
11. Introduce the validated route and content registry.
12. Add Explorations and Community with truthful states.
13. Reorder homepage proof and make publishable work navigable.
14. Build and publish the complete TEDx RSU case study.
15. Restore CV delivery and audit claims.
16. Implement the contact endpoint, form states and privacy update.
17. Consolidate CSS and semantic tokens without visual redesign.
18. Complete accessibility, responsive, performance and security QA.
19. Create a preview deployment for owner review.
20. Merge and deploy only after explicit visual approval.

Explicit non-goals

- Recreating the rejected thesis-aligned redesign.
- Replacing the current visual language with Material 3 styling.
- Inventing project results, testimonials, community entries or working-principles copy.
- Publishing incomplete case studies as if finished.
- Adding a database, account system, CMS or analytics platform without demonstrated need.
- Deploying visual changes before review.

Implementation workflow

This plan is deliberately sequenced to protect the restored visual baseline. Each task creates an independently reviewable result and ends with verification before the next structural layer begins.

> ***For agentic workers: REQUIRED SUB-SKILL: Use superpowers:subagent-driven-development (recommended) or superpowers:executing-plans to implement this plan task-by-task. Steps use checkbox (- [ ]) syntax for tracking.***

Goal: Strengthen the restored portfolio's proof, navigation, accessibility, conversion and engineering quality without changing its approved visual character.

Architecture: Keep the static HTML, CSS and ES-module platform. Add validated build-time content domains and one narrow Vercel contact endpoint. Prerender essential content and use JavaScript only for enhancement.

Tech Stack: HTML, CSS, ES modules, Node.js build scripts and tests, Vercel Functions, Resend, infrastructure-backed rate limiting, Playwright and Lighthouse.

Spec: `docs/superpowers/specs/2026-10-13-portfolio-design-engineering-review.md`

Global Constraints

- Commit `54dcc41` is the visual baseline.
- Preserve the current palette, typography, spacing character, imagery and restrained motion.
- Do not invent content, outcomes, testimonials, collaborators or community entries.
- Core navigation and narrative must work without JavaScript.
- Work on a `codex/` branch and use a preview deployment; do not merge to `main` without explicit approval.
- Maintain WCAG 2.2 AA, 320px support and the performance budgets in the specification.

Task 1: Freeze the approved visual baseline

Files:

- Create: `tests/visual-contract/route-contract.mjs`
- Create: `tests/visual-contract/README.md`
- Modify: `package.json`

Interfaces:

- Produces: `npm run test:contract`, a non-screenshot contract check for protected visual and structural primitives.

☐ Record the protected route, font, token, image-treatment and footer-signature assertions from the specification.

☐ Add failing assertions for the visual baseline and required structural changes.

☐ Run `npm run test:contract` and confirm failures describe only missing new contracts.

☐ Add the script to `package.json` and make `npm test` run it after the existing integrity suite.

☐ Run the full suite and commit with `test: freeze approved portfolio visual contract`.

Task 2: Replace obsolete integrity contracts

Files:

- Modify: `scripts/site-integrity.test.mjs`
- Modify: `data/site.js`

Interfaces:

- Produces: tests that allow mixed `published` and `coming-soon` project states.

☐ Replace the assertion that every project is locked with explicit state validation.

☐ Add failing tests requiring one published flagship and truthful unfinished states.

☐ Update project data minimally so TEDx RSU is the only initial published flagship.

☐ Run `npm test` and confirm the new state contract passes without changing presentation.

☐ Commit with `test: align project publication contract`.

Task 3: Introduce validated content domains

Files:

- Create: `data/site-config.js`
- Create: `data/navigation.js`
- Create: `data/projects.js`
- Create: `data/case-studies.js`
- Create: `data/services.js`
- Create: `data/writing.js`
- Create: `scripts/validate-content.mjs`
- Modify: `scripts/build-site.mjs`
- Modify: `package.json`

Interfaces:

- Produces: `validateContent(): { routes, projects, caseStudies, services, writing }`.
- Validates: unique slugs, publication states, route ownership, media dimensions, next-project references and required metadata.

☐ Write fixture-based failing tests for duplicate slugs, missing dimensions and invalid project references.

☐ Implement the validator with explicit error messages.

☐ Split public data without changing rendered output.

☐ Generate the route manifest and sitemap from validated data.

☐ Run `npm test` and compare the production HTML output with the baseline except for intentional metadata ordering.

☐ Commit with `refactor: validate portfolio content domains`.

Task 4: Add Explorations and Community routes

Files:

- Create: `explorations.html`
- Create: `community.html`
- Modify: `data/navigation.js`
- Modify: `js/render/navigation.js`

- Modify: `js/render/footer.js`
- Modify: `scripts/build-site.mjs`
- Modify: `sitemap.xml`
- Modify: `styles/pages-v4.css`

Interfaces:

- Produces: prerendered routes with truthful empty or real-entry states.

☐ Write failing route, metadata, navigation and no-JavaScript content tests.

☐ Add the two routes using existing page-shell and typography patterns.

☐ Add destinations to the full Index and footer without crowding the persistent bar.

☐ Add only the minimum route-specific CSS needed.

☐ Verify at 320px, 390px and 1440px with both themes.

☐ Commit with `feat: add explorations and community routes`.

Task 5: Strengthen navigation without changing its character

Files:

- Modify: `js/render/navigation.js`
- Modify: `styles/components-v4.css`
- Modify: `styles/pages-v4.css`
- Modify: root HTML files for prerendered fallback links

Interfaces:

- Produces: `openIndex(trigger)`, `closeIndex({ restoreFocus })` and complete static fallback navigation.

☐ Write failing tests for Index destinations, Escape handling, focus containment and focus restoration.

☐ Implement the editorial Index using existing type, colour and spacing roles.

☐ Keep the persistent desktop and compact controls visually quiet.

☐ Verify all destinations remain available without JavaScript.

☐ Complete keyboard, touch and reduced-motion checks.

☐ Commit with `feat: strengthen portfolio index navigation`.

Task 6: Reorder homepage proof and repair project entry points

Files:

- Modify: `index.html`
- Modify: `js/render/projects.js`
- Modify: `js/render/sections.js`
- Modify: `styles/pages-v4.css`

Interfaces:

- Consumes: validated project publication state.
- Produces: accessible project anchors for published work and truthful disabled states for unfinished work.

☐ Write failing tests for the approved homepage order and published-project links.

☐ Move Selected Work before the profile section without changing section styling.

☐ Render published projects as anchors and unfinished projects as non-interactive entries.

☐ Gate hover transforms behind `(hover: hover) and (pointer: fine)` and provide equivalent focus treatment.

☐ Verify geometry, reading order and focus order at every required width.

☐ Commit with `feat: put portfolio proof before biography`.

Task 7: Publish the TEDx RSU flagship case study

Files:

- Modify: `data/case-studies.js`
- Modify: `work/tedx-rsu.html`
- Modify: `js/render/case-markup.js`
- Modify: `js/render/case-study.js`
- Modify: `styles/pages-v4.css`
- Add: supplied TEDx media under `assets/images/projects/tedx-rsu/`

Interfaces:

- Produces: `renderCaseStudy(caseStudy)` supporting facts, narrative chapters, media and next-project transition.

☐ Add a failing structural test for proposition, facts, required narrative chapters, media dimensions and next project.

☐ Populate only verified owner-supplied content and omit unavailable outcomes.

☐ Prerender the complete narrative before JavaScript enhancement.

☐ Add restrained chapter orientation that does not resemble application navigation.

☐ Verify media loading, alt text, focus, no-JavaScript use and responsive measure.

☐ Commit with `feat: publish TEDx RSU flagship case study`.

Task 8: Restore CV delivery and audit evidence claims

Files:

- Add: `assets/documents/miebaka-joshua-cv.pdf`
- Modify: `about.html`
- Modify: `js/render/navigation.js`
- Modify: `data/site-config.js`
- Modify: `scripts/site-integrity.test.mjs`

Interfaces:

- Produces: stable `DOWNLOAD CV` link with PDF type and file size.

☐ Verify the approved resume source and copy it to the stable public filename.

☐ Add failing tests for file presence, link text, MIME type and size metadata.

☐ Add the action to About and the Index without adding a top-level destination.

☐ Audit every numerical claim and remove or qualify unsupported claims.

☐ Run link and content verification.

☐ Commit with `feat: restore verified CV delivery`.

Task 9: Implement the contact service boundary

Files:

- Create: `api/contact.js`
- Create: `api/lib/contact-schema.js`
- Create: `api/lib/rate-limit.js`
- Create: `api/lib/mail-service.js`
- Create: `api/lib/resend-mail-service.js`
- Create: `tests/contact-api.test.mjs`
- Modify: `vercel.json`
- Modify: `package.json`

Interfaces:

- Consumes: `{ name: string, email: string, message: string, interest?: string, website?: string, startedAt?: number }`.
- Produces: `{ ok: true, requestId: string }` or `{ ok: false, code: string, requestId: string, retryAfter?: number }`.

☐ Write failing schema tests for valid input, blank message, excessive lengths, malformed JSON, unexpected values and oversized bodies.

☐ Implement the pure schema validator.

☐ Define the mail-service interface and mock it in tests.

☐ Implement same-origin checks, accessible abuse signals and infrastructure-backed rate limiting.

☐ Implement the Resend adapter with `AbortSignal.timeout(8000)`.

☐ Add privacy-safe structured logs and stable error responses.

☐ Run unit and integration tests without contacting the real provider.

☐ Commit with `feat: add secure contact delivery endpoint`.

Task 10: Upgrade the contact client and privacy contract

Files:

- Modify: `contact.html`
- Modify: `js/core/form.js`
- Modify: `privacy.html`
- Modify: `styles/components-v4.css`
- Create: `tests/contact-form.test.mjs`

Interfaces:

- Consumes: the contact endpoint response envelope from Task 9.
- Produces: submitting, success, validation, rate-limit and provider-failure states while preserving values.

☐ Write failing tests for required message, merged `aria-describedby`, double-submit prevention, retained values and focus on first error.

☐ Implement client validation as a convenience while keeping server validation authoritative.

☐ Submit to `/api/contact` with an abort timeout and explicit recovery messages.

☐ Preserve visible email and a truthful no-JavaScript fallback.

☐ Update privacy copy for fields, purpose, processors and operational retention.

☐ Commit with `feat: complete accessible contact flow`.

Task 11: Repair motion, accessibility and the client marquee

Files:

- Modify: `js/render/projects.js`
- Modify: `styles/foundations.css`
- Modify: `styles/components-v4.css`
- Modify: `styles/pages-v4.css`
- Modify: `scripts/site-integrity.test.mjs`

Interfaces:

- Produces: user-controlled marquee state and purpose-specific reduced motion.

☐ Write failing tests for Pause or Play control semantics and reduced-motion static rendering.

☐ Add the visible control and persist its state only for the current visit.

☐ Replace the global one-millisecond animation kill with component-specific reduced-motion rules.

☐ Expand small interactive hit regions to a practical 44 by 44 pixels without visual bulk.

☐ Verify keyboard, touch, forced colours, text spacing and 200 percent zoom.

☐ Commit with `fix: complete accessible motion controls`.

Task 12: Consolidate tokens and CSS ownership

Files:

- Modify: `styles/foundations.css`
- Modify: `styles/components-v4.css`
- Modify: `styles/pages-v4.css`
- Modify: `styles/main.css`
- Modify: `scripts/site-integrity.test.mjs`

Interfaces:

- Produces: one bundled production stylesheet with semantic interface colour usage and reduced selector duplication.

☐ Add tests that reject raw interface colours outside reference-token declarations and approved project media.

☐ Inventory repeated selectors and assign each to one owning source section.

☐ Remove obsolete override strata in small, reviewable groups.

☐ Confirm computed styles against the baseline after every group.

☐ Verify compressed CSS stays below 45KB and no route regresses.

☐ Commit with `refactor: consolidate portfolio style contracts`.

Task 13: Add deployment and browser quality gates

Files:

- Create: `.github/workflows/quality.yml`
- Create: `tests/e2e/portfolio.spec.mjs`
- Create: `tests/e2e/contact.spec.mjs`
- Create: `lighthouse.config.cjs`
- Modify: `package.json`
- Modify: `vercel.json`

- Modify: `netlify.toml`

Interfaces:

- Produces: `npm run test:e2e` and `npm run test:lighthouse`.

☐ Add CI jobs for build, unit, integrity and dependency audit checks.

☐ Add route checks at 320px, 390px, tablet, 1280px and 1440px.

☐ Assert no horizontal overflow, broken assets, duplicate IDs or console errors.

☐ Test Index keyboard behavior and contact success and failure with mocked provider responses.

☐ Enforce LCP, INP, CLS, CSS, JavaScript and homepage-media budgets.

☐ Make Vercel canonical and test header parity if Netlify remains supported.

☐ Commit with `test: add portfolio release quality gates`.

Task 14: Preview review and release

Files:

- Modify only files required by verified defects found during final QA.

Interfaces:

- Produces: approved preview deployment and releasable branch.

☐ Run `npm test`, `npm audit --audit-level=moderate`, `npm run test:e2e`, `npm run test:lighthouse` and `git diff --check`.

☐ Inspect every route, both themes, reduced motion, forced colours, touch, keyboard and JavaScript-disabled content.

☐ Verify the preview deployment headers, API behavior, privacy copy, performance and asset integrity.

☐ Present the preview and named material changes for owner approval.

☐ Merge only after explicit approval.

☐ Verify the production deployment against the approved commit.

☐ Use `superpowers:finishing-a-development-branch` to close the branch safely.